

Day 2 Lab: Build a Business Dashboard

Duration: 90 minutes | **Difficulty:** Beginner → Intermediate

Prerequisites: Claude Code installed, authenticated, Node.js 18+, Git installed

© 2026 AIA Copilot — Instructor-Led Training Material

Overview

In this lab, you'll build **one complete application from scratch** — a Business Dashboard with real-time metrics, task management, and a SQLite database. Every exercise adds a layer to the same project, so by the end of the day you'll have a working app you built entirely with Claude Code.

No SQL server required. Everything runs locally with SQLite — zero configuration.

What you'll build: - A REST API with 10+ endpoints - A SQLite database with automatic seeding - A live web UI showing metrics and tasks as a dark-themed dashboard - Data visualization with charts and filters - CSV export functionality - Git version control with clean conventional commits

What you'll master: - The agentic loop: how Claude plans, executes, and validates autonomously - CLAUDE.md as **policy, not documentation** — how rules change behavior - @ mentions for precision context feeding (and why broad prompts cost more) - Recovery prompting — fixing things when they break - Iterative refinement: the "make it better" workflow - Branch-based Git workflows in natural language

Pre-Flight Check (5 min)

Before we begin, verify your environment. Run each command and confirm you get a version number:

1. Check Node.js

```
node -v
```

Expected output: `v18.x.x` or higher

2. Check npm

```
npm -v
```

Expected output: `9.x.x` or higher

3. Check Git

```
git --version
```

Expected output: `git version 2.x.x` (any recent version)

4. Check Claude Code

```
which claude
```

Expected output: A path like `/usr/local/bin/claude` or similar

If not installed:

```
# Recommended (auto-updates)
curl -fsSL https://claude.ai/install.sh | bash

# Or via Homebrew
brew install --cask claude-code

# Or via WinGet (Windows)
winget install Anthropic.ClaudeCode
```

5. Check Port Availability

Port 3100 must be available (we avoid 3000 because other tools often claim it):

```
curl http://localhost:3100 2>/dev/null && echo "PORT 3100 IN USE – pick another" || echo "PORT 3100 AVAILABLE ☐"
```

If 3100 is taken, use any open port (3200, 3300, etc.) and substitute it throughout the lab.

⚠ **Windows users:** Use WSL or Git Bash for this lab. PowerShell quoting will break the curl examples. If you see errors with single quotes in curl commands, you're in the wrong shell.

☐ Pre-Flight Complete

- ☐ Node.js 18+ installed
 - ☐ npm available
 - ☐ Git installed
 - ☐ Claude Code CLI installed and on PATH
 - ☐ Port 3100 available
 - ☐ Running in bash/zsh (not PowerShell)
-

Exercise 1: The Magic Moment (20 min)

Goal

Go from an empty folder to a running web app in a single prompt. This is the moment that changes how you think about building software.

Steps

1. Create Your Project

```
mkdir business-dashboard  
cd business-dashboard  
git init
```

2. Start Claude Code

```
claude
```

You're now in an interactive session. The prompt shows your working directory.

3. The Build Prompt

Copy and paste this prompt exactly:

Build me a business dashboard web application:

Tech stack: Node.js, Express, SQLite (via better-sqlite3), vanilla HTML/CSS/JS frontend
No external CSS frameworks – write clean, modern CSS

Database tables:

1. metrics – id, name, value (number), category, recorded_at
2. tasks – id, title, status (pending/in-progress/completed), priority (high/medium/low), assigned_to, created_at, completed_at

API endpoints:

- GET /api/metrics (filter by ?category=)
- POST /api/metrics
- GET /api/tasks (filter by ?status= and ?priority=)
- POST /api/tasks
- PUT /api/tasks/:id
- DELETE /api/tasks/:id
- GET /api/dashboard (summary: all metrics + task counts by status + high priority tasks)

Frontend: Dark theme dashboard at / showing all metrics as cards and tasks as a list.
Seed the database with 8 sample metrics across categories (financial, customers, operations, hr) and 5 sample tasks.

Initialize npm, install dependencies, create everything. Use port 3100 (or PORT env var).

Watch Claude work through the agentic loop: 1. **Plan** — Analyzes requirements, decides on file structure 2. **Execute** — Creates files, runs `npm install`, writes code 3. **Validate** — Checks for errors, tests the server

***Instructor Note:** This is the "magic moment." Students see Claude build a complete, working application from a single prompt. Let it run — don't interrupt.*

4. Test the Dashboard API

Once Claude reports it's done, open a **new terminal** (keep Claude session running):

```
curl http://localhost:3100/api/dashboard | python3 -m json.tool
```

Expected output: A JSON object with metrics, task counts by status, and high-priority tasks.

5. See It in the Browser

Open your browser to `http://localhost:3100`

You should see a dark-themed dashboard with 8 metric cards and 5 tasks.

6. Test the API Endpoints

```
# Add a new metric
curl -X POST http://localhost:3100/api/metrics \
  -H 'Content-Type: application/json' \
  -d '{"name":"New Leads This Week","value":42,"category":"sales"}'

# Add a new task
curl -X POST http://localhost:3100/api/tasks \
  -H 'Content-Type: application/json' \
  -d '{"title":"Review Q2 projections","priority":"high","assigned_to":"You"}'

# Complete a task
curl -X PUT http://localhost:3100/api/tasks/1 \
  -H 'Content-Type: application/json' \
  -d '{"status":"completed"}'
```

Refresh the dashboard in your browser. The new data should appear.

What Just Happened?

This is the **agentic loop** in action:

1. **Planning:** Claude read your prompt, understood the requirements, and created a mental plan
2. **Execution:** It autonomously created files, ran npm commands, wrote code
3. **Validation:** It checked that the structure made sense before presenting it to you

You didn't tell Claude *how* to build it. You told it *what* you wanted. Claude: - Knew to use Express (industry standard for Node.js APIs) - Chose better-sqlite3 (fast, zero-config database) - Structured the code properly (server setup, routes, database layer) - Added error handling without being asked - Seeded test data so the dashboard isn't empty - Built a complete frontend with dark theme styling

This is fundamentally different from autocomplete or code snippets. Claude understood the **intent** and delivered a **working system**.

Challenge

Let's see the difference between vague and specific prompts. Try this vague one first:

```
Add validation.
```

Look at what Claude does — it guesses which routes, which fields, what error format.

Now try the specific version:

```
In @src/index.js, modify only the POST /api/tasks route. Validate:
- title: must be non-empty string, max 200 characters
- priority: must be one of "high", "medium", "low"
- assigned_to: optional, but if provided must be non-empty string
Return 400 with { error: "description of what's wrong" } for invalid requests.
```

Compare the outputs. **The specific prompt produces exactly what you want on the first try.** The vague prompt requires 2-3 rounds of correction.

Teaching point: *Specificity isn't extra work — it's less work. One precise prompt beats three vague ones.*

7. Your First Git Commit

In the Claude Code session:

```
Initialize git, create a .gitignore for node_modules and *.db files,
then commit everything with the message "feat: initial business dashboard with API and
UI"
```

☐Checkpoint

You should now have: - [] A running Express server on port 3100 - [] SQLite database with metrics and tasks - [] Working web UI at <http://localhost:3100> - [] Input validation on POST /api/tasks - [] 7 API endpoints (all testable with curl) - [] First git commit

Exercise 2: Add Your Project Brain — CLAUDE.md (15 min)

Goal

Teach Claude your team's conventions so every future change follows your standards automatically — without you asking.

Key insight: CLAUDE.md is policy, not documentation. It doesn't describe your project — it controls Claude's behavior.

Steps

1. Clear Context

In your Claude Code session:

```
/clear
```

This starts a fresh context window. The project files are still on disk — Claude just starts with a clean conversation.

2. Create CLAUDE.md

Tell Claude:

```
Create a CLAUDE.md file in the project root with these conventions:

# Business Dashboard - Project Conventions

## Tech Stack
- Node.js + Express backend
- SQLite via better-sqlite3 (no external DB required)
- Vanilla HTML/CSS/JS frontend (no frameworks)
- Dark theme UI: background #0f172a, cards #1e293b, borders #334155

## Code Style
- Use const/let, never var
- Always prefer async/await over callbacks
- Return early for validation failures (guard clauses)
- All routes must use try/catch with proper error responses
- All API responses return JSON
- Error responses include { error: "message" }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found
- SQL uses parameterized queries (never string concatenation)

## Git Conventions
- Conventional commits: feat:, fix:, docs:, refactor:
- One logical change per commit
- No committing node_modules or .db files

## File Structure
- src/ for server code
- public/ for frontend files
- Database file: dashboard.db (auto-created on first run)

## Testing
- Test API endpoints with curl before marking complete
- Verify frontend loads and displays data after changes
```

3. Verify Claude Reads It

Start a new context to pick up the CLAUDE.md:

```
/clear
```

Then ask:

```
What conventions does this project follow?
```

Expected output: Claude recites your rules back to you — code style, response format, Git conventions, file structure. This confirms CLAUDE.md is loaded.

4. Test Convention Enforcement

Now ask Claude to make a change:

```
Add a GET /api/tasks/stats endpoint that returns:  
- total tasks count  
- count by status  
- count by priority  
- average completion time (for completed tasks)
```

Watch Claude follow your conventions automatically: - Uses `const` (not `var`) — your code style rule - Returns early for edge cases — your guard clause rule - Wraps in try/catch — your error handling rule - Returns proper JSON with error handling — your API rule - Uses parameterized SQL — your security rule

5. Verify the New Endpoint

```
curl http://localhost:3100/api/tasks/stats | python3 -m json.tool
```

Expected output: JSON with total count, status breakdown, priority breakdown, and average completion time.

What Just Happened?

CLAUDE.md is magic. Here's why:

Every time Claude starts working, it automatically: 1. Looks for `CLAUDE.md` in the project root 2. Reads it before doing anything else 3. Treats it as the source of truth for your project

You wrote ONE file with your conventions, and now: - Every endpoint Claude creates follows your response format - Every function uses `async/await` - Every route has try/catch - File placement matches your structure

Without CLAUDE.md: You'd have to remind Claude every time: "Use async/await. Make sure error handling is consistent. Follow this response format..."

With CLAUDE.md: Claude just knows.

This is how professional teams use Claude Code. CLAUDE.md becomes your team's shared brain — new developers (human or AI) read it and instantly understand your standards.

***The mental shift:** You didn't ask for try/catch. You didn't ask for guard clauses. CLAUDE.md made those automatic. That's the difference between a prompt and a policy.*

6. Commit

```
Commit this change following our git conventions
```

Claude should use `feat: add task statistics endpoint` or similar.

Challenge

Add a new convention to CLAUDE.md:

```
## Logging
- All routes must log request method, path, and response time to console
- Format: [timestamp] METHOD /path - STATUS (XXXms)
```

Save the file, then ask Claude:

```
Add a GET /api/health endpoint that returns the server status and uptime
```

Check the response. Does it include logging? Claude should follow the new convention immediately!

☐Checkpoint

- ☐ CLAUDE.md file exists in project root
- ☐ Claude follows your conventions automatically (const, try/catch, guard clauses)
- ☐ Stats endpoint works: `curl http://localhost:3100/api/tasks/stats`
- ☐ Clean conventional commit

Exercise 3: Context Feeding with @ Mentions (15 min)

Goal

Master context control with @ mentions to make Claude look exactly where you want — and understand why precision matters for cost and quality.

Steps

1. Context Awareness Demo

Before using @ mentions, try this broad prompt:

```
Update the API to handle errors better.
```

Notice: Claude has to search your entire project, decide which files matter, and guess what "better" means. This burns tokens and produces generic results.

Now try with precision:

```
Look at @src/index.js – the routes are all in one file.  
Refactor: extract metric routes into src/routes/metrics.js  
and task routes into src/routes/tasks.js.  
Keep the main file clean with just app setup, middleware, and route mounting.
```

Watch Claude: 1. Read the specific file you mentioned 2. Identify the route groups 3. Create two new files following your CLAUDE.md conventions 4. Update the main file to import and mount them 5. Test that nothing broke

Why this matters: Every token Claude reads costs money and uses limited context window space. @ mentions load exactly the files you need — nothing more. In a large codebase, the difference between `@src/auth.ts` and "look at the auth code" could be 50,000 tokens of unnecessary context.

2. Verify Nothing Broke

In a separate terminal:

```
# Test metrics
curl http://localhost:3100/api/metrics | python3 -m json.tool

# Test tasks
curl http://localhost:3100/api/tasks | python3 -m json.tool

# Test dashboard
curl http://localhost:3100/api/dashboard | python3 -m json.tool
```

Expected output: All three endpoints return valid JSON. The refactor was invisible to the API consumer.

3. Intentional Break & Recovery

This is how developers actually use Claude Code — things break, and you fix them.

Manually delete one of the route files you just created:

```
# In a separate terminal:
rm src/routes/metrics.js
```

Now go back to Claude Code:

```
The metrics routes file was deleted. Restore it based on the original
API spec and our CLAUDE.md conventions. Make sure all metric endpoints work.
```

Watch Claude: 1. Reads the remaining code to understand what's missing 2. Checks CLAUDE.md for conventions 3. Rebuilds the file following your rules 4. Tests that it works

What Just Happened?

Recovery prompting is a core skill. Claude's diff awareness means it can reconstruct what's missing by analyzing what's still there.

Think about what happened: - Claude read `src/routes/tasks.js` to understand the pattern - It checked `src/index.js` to see what routes were being imported - It regenerated the metrics routes following the same conventions - It verified everything still works

In the real world, you'll use this pattern when: - A merge conflict corrupts a file - You accidentally delete something - A dependency upgrade breaks existing code - You need to reverse-engineer a missing component

The pattern: Describe what's wrong + what the result should look like. Claude handles the detective work.

4. Add a Feature Using Multiple File References

```
Look at @src/routes/tasks.js and @public/index.html – add the ability
to create a new task from the frontend UI. Add a simple form at the top
of the tasks section with fields for title, priority (dropdown), and
assigned_to. When submitted, POST to the API and refresh the task list.
```

Watch Claude: 1. Read both files to understand the current structure 2. Add a form to the HTML with proper styling 3. Write JavaScript to handle form submission 4. POST to the existing API endpoint 5. Refresh the task list on success

5. Test the Form

Open `http://localhost:3100` in your browser. You should see a form above the task list. Create a task and verify it appears.

6. Commit

```
Commit these changes as two separate commits:
one for the refactor, one for the new feature
```

Watch Claude: - Run `git status` and `git diff` - Organize changes into logical groups - Write descriptive conventional commit messages for each

Challenge

Try using @ mentions with a URL to improve your code:

```
@https://expressjs.com/en/guide/error-handling.html Update our error
handling middleware to match Express best practices from this guide
```

Claude will fetch the Express documentation, read the patterns, and update your code to match official best practices.

☐Checkpoint

- ☐ Routes split into separate files (src/routes/)
- ☐ Recovery from deleted file successful
- ☐ All API endpoints still work
- ☐ Frontend has a "New Task" form that creates tasks
- ☐ Two clean git commits (refactor + feature)

Exercise 4: Iterative Refinement (20 min)

Goal

Experience the "iterate to great" workflow — start rough, polish progressively. This is how professionals use Claude Code day-to-day.

Steps

1. Add Data Visualization

```
Add a simple bar chart to the dashboard showing task counts by status
(pending, in-progress, completed). Use vanilla JavaScript and CSS —
no chart libraries. Make it look good with our dark theme colors.
```

Watch Claude: - Create CSS-based bars (no external library needed) - Match your dark theme from CLAUDE.md - Fetch data from the existing `/api/tasks/stats` endpoint - Render the chart on the dashboard

2. Verify in Browser

Open `http://localhost:3100` and look at the chart.

3. Critique and Refine

Look at the result, then tell Claude what to improve:

```
The chart works but [describe what you'd change — colors, spacing, labels, etc.]
Make it better.
```

***This is the key skill.** You don't need to know how to fix it. You just need to know what "better" looks like and describe it. Claude handles the implementation.*

Try 2-3 rounds of refinement. Each time, describe what's not right and let Claude fix it.

4. Add Search and Filter

```
Add a filter bar above the tasks list:
- Dropdown to filter by status (All, Pending, In Progress, Completed)
- Dropdown to filter by priority (All, High, Medium, Low)
- Filters should update the list immediately (no page reload)
```

5. Test the Filters

In your browser, change the dropdowns. The task list should update instantly without a page refresh.

6. Production Hardening

Before polishing the UI further, ask Claude an architectural question:

```
What would you change about this codebase before deploying to production?
```

Expected output: Claude suggests environment variables, logging, rate limiting, CORS, security headers, input sanitization, and more.

Review the suggestions, then:

```
Implement the top 3 most important production improvements you just suggested.
```

What Just Happened?

You just used **Claude Code as an architectural partner**, not just a code generator.

The "what would you change?" pattern is incredibly powerful because: - Claude has seen thousands of production deployments in its training - It knows the difference between "demo code" and "production code" - It applies improvements to YOUR specific codebase, not generic advice

The iterate-to-great workflow: 1. Build something that works (Exercise 1) 2. Add structure and conventions (Exercise 2) 3. Refactor for quality (Exercise 3) 4. Polish iteratively (Exercise 4)

Each round makes the code better without starting over. This is exactly how professional development works — no one ships the first draft.

***The deeper insight:** Claude Code isn't just a code generator. It's four things at once: - A **code generator** (writes files from specs) - A **project memory engine** (CLAUDE.md as persistent policy) - A **context navigator** (@ mentions for precision) - An **architectural advisor** ("what would you change?")*

Most people only use the first one. You now use all four.

7. Polish the UI

```
Look at @public/index.html — the dashboard needs these improvements:
```

1. Add a last-updated timestamp that refreshes every 30 seconds
2. Metrics cards should show a colored indicator: green for positive trends, red for issues
3. The task list should show how long ago each task was created (e.g., "2 hours ago")
4. Add subtle hover effects on cards and task items

8. Commit

```
Commit all improvements with message "feat: add chart, filters, production hardening, and UI polish"
```

Challenge

Try the "interview me" pattern:

```
I want to add user authentication to this dashboard. Before writing any code, interview me: ask me 5 questions about my requirements so you build exactly what I need.
```

Answer Claude's questions, then let it build the auth system based on your answers. Notice how much better the result is when Claude asks first.

☐ Checkpoint

- ☐ Bar chart showing task status distribution
- ☐ Working filter dropdowns (status + priority)
- ☐ Production improvements applied (env vars, logging, security headers, etc.)
- ☐ Auto-refreshing timestamp
- ☐ Polished, professional-looking dashboard
- ☐ Clean git history with 4+ commits

Exercise 5: The Git Workflow (20 min)

Goal

Use Claude Code for branch-based development like a real team — branching, committing, and merging all in natural language.

Steps

1. Create a Feature Branch

```
Create a new git branch called "feature/export" and switch to it
```

2. Build the Export Feature

Add a CSV export feature:

1. New endpoint: GET /api/export/metrics?format=csv returns metrics as downloadable CSV
2. New endpoint: GET /api/export/tasks?format=csv returns tasks as downloadable CSV
3. Add "Export CSV" buttons to the dashboard UI for both metrics and tasks
4. Buttons should trigger file downloads

Watch Claude: - Create the export routes following your CLAUDE.md conventions - Set proper Content-Type and Content-Disposition headers for downloads - Add styled buttons to the frontend matching the dark theme - Wire up the button click handlers to trigger downloads

3. Test the Export

In a separate terminal:

```
# Test metrics export
curl http://localhost:3100/api/export/metrics?format=csv
```

Expected output: CSV formatted data with headers like `id,name,value,category,recorded_at`

```
# Test tasks export
curl http://localhost:3100/api/export/tasks?format=csv
```

Expected output: CSV formatted data with task fields.

Also test the buttons in your browser — they should download .csv files when clicked.

4. Commit and Merge

Commit the export feature, switch back to main, and merge the feature branch

Watch Claude: 1. Stage all changes 2. Write a descriptive commit message 3. Switch to main branch 4. Merge the feature branch 5. Optionally delete the feature branch

5. Review Your Git History

Show me the git log with a nice one-line format

Expected output: 6-7 clean conventional commits telling the story of your project:


```
abc1234 feat: add CSV export for metrics and tasks
def5678 feat: add chart, filters, production hardening, and UI polish
ghi9012 feat: add task creation form to frontend
jkl3456 refactor: extract routes into separate files
mno7890 feat: add task statistics endpoint
pqr1234 feat: initial business dashboard with API and UI
```

What Just Happened?

You just experienced **Git workflow automation in natural language**.

Traditional workflow:

```
git checkout -b feature/export
# Write code manually
git add .
git commit -m "add export" # Lazy message
git checkout main
git merge feature/export
```

Claude Code workflow:

Create a feature branch, build export, commit, and merge back to main

Why this matters:

1. **No context switching:** Stay in natural language, don't drop to Git commands
2. **Better commit messages:** Claude writes conventional commits with descriptive bodies
3. **Git best practices:** Claude follows proper workflows (feature branches, descriptive commits)
4. **The entire workflow is auditable:** Your Git log tells a clear story

***The deeper insight:** Claude doesn't just write code. It understands the **entire development workflow** — planning, coding, testing, version control, documentation. You orchestrate; Claude executes.*

Challenge

Ask Claude to do a code review of your project:

```
Review the entire codebase against our CLAUDE.md conventions.
Are there any violations? List them and fix the top 3.
```

Claude will audit every file against your rules and fix what it finds.

□ Final Checkpoint

Your completed Business Dashboard includes: - [] REST API with 10+ endpoints - [] SQLite database (zero configuration) - [] Professional dark-themed web UI - [] Input validation with proper error responses - [] Bar chart visualization - [] Task filters (status + priority) - [] CSV export for metrics and tasks - [] Add-task form on the frontend - [] Production hardening (env vars, security headers, logging) - [] CLAUDE.md with team conventions - [] Clean git history with conventional commits - [] Feature branch workflow

What You Built vs. What You Typed

Count your prompts. You probably typed **fewer than 20 messages** to Claude Code and got:

- ~500 lines of backend code
- ~200 lines of frontend code
- A working SQLite database with sample data
- A professional UI with charts, filters, and forms
- Production-quality error handling and security
- Clean git history with 6+ conventional commits

That's the leverage. You described what you wanted. Claude Code built it.

Key Takeaways

- 1. The Agentic Loop** - Claude doesn't just complete code — it plans, executes, and validates - You provide intent, Claude provides implementation - This is fundamentally different from autocomplete
- 2. CLAUDE.md is Your Team Brain** - One file that teaches Claude your conventions - Automatic adherence to standards — no reminding - Living documentation that AI and humans both read
- 3. Context Control with @ Mentions** - @ mentions = surgical precision for Claude's attention - Focus Claude's attention, save tokens, get better results - Bring in external docs on-demand with URL @ mentions

4. Iterate to Great - Start rough, polish progressively - "What would you change?" turns Claude into an architect - Each round makes the code better without starting over

5. Natural Language Git - Branching, committing, merging — all in plain English - Better commit messages than most humans write - Your Git log tells a clear story

Reflection (5 min)

Take a moment to answer these questions — write your answers down or discuss with a neighbor:

1. **Where did Claude surprise you?** (Something it did better than expected)
 2. **Where did it fail or need correction?** (What went wrong and how did you fix it)
 3. **What single prompt made the biggest difference?** (The one that produced the most value)
 4. **When would you NOT use Claude Code?** (What tasks are better done manually)
-

☐ Continues in Day 3

Tomorrow, you'll take this exact dashboard and add: - Custom slash commands for common operations - Security hooks (secret detection, audit logging) - A report-generation skill - MCP integration for live data - Deployment with subagents working in parallel

Don't delete this project. You'll need it tomorrow.

Next Steps

Practice Tonight: - Add a new feature to your dashboard (e.g., task comments, metric history charts) - Try modifying CLAUDE.md and see how behavior changes - Experiment with `/compact` to manage long sessions

Pre-Read for Day 3: - Think about what repetitive tasks you'd like to automate with custom commands - Consider what external data sources your dashboard could connect to - Read about MCP (Model Context Protocol) if you want a head start

Troubleshooting

Common Issues and Solutions

Issue	Fix
Port 3100 in use	Set <code>PORT=3200</code> before running: <code>PORT=3200 node src/index.js</code>
Port 3000 in use	We use 3100 by default. If anything says 3000, change it to 3100.
<code>better-sqlite3</code> won't install	Run <code>npm rebuild better-sqlite3</code> or try <code>npm install --build-from-source</code>
Database locked	Stop any running server instances: <code>pkill -f "node src/index"</code>
Claude seems confused	Run <code>/clear</code> and re-state what you want with file references
Git conflicts	Ask Claude: "resolve any git conflicts and show me the result"
PowerShell curl errors	Switch to WSL or Git Bash. PowerShell handles quotes differently.
Node version too old	Run <code>node -v</code> — must be 18+. Use <code>nvm install 18</code> if needed.
Claude Code not found	Run <code>curl -fsSL https://claude.ai/install.sh \& bash</code> then restart terminal.
Slow responses	Try <code>/model sonnet</code> for faster (but slightly less capable) responses.
Context too large	Run <code>/compact</code> to compress conversation history.
Claude not following conventions	Verify <code>CLAUDE.md</code> is in project root. Try <code>/clear</code> so Claude re-reads it.

Generated from Day2_Lab_Business_Dashboard.md on Day2_Lab_Business_Dashboard.pdf

© 2026 AIA Copilot Training — Claude Code Fundamentals